

Technical Report

[Link the notebook](#)

Introduction

Stock market prediction is a challenging task because financial time series are noisy, non-stationary, and influenced by a wide range of macroeconomic, behavioral, and technical factors. In this project, the goal was to build machine learning models that use historical market data and technical indicators to classify whether a stock should generate a buy, sell, or hold signal. The data used consisted of historical stock price records and company metadata, including ticker, date, open/high/low/close prices, adjusted close, volume, sector, and industry information. The central objective was to determine whether engineered technical indicators such as the Moving Average Convergence Divergence (MACD) and the Relative Strength Index (RSI) could be translated into actionable trading signals and predicted reliably by supervised learning algorithms.

Data Preparation

The initial dataset was merged using the stock price data and stock metadata on the ticker identifier. The records were sorted chronologically by stock and date to preserve time-series integrity before computing indicator values. This step was essential because financial indicators rely on prior observations and should not be calculated across inconsistent time ordering.

Data cleaning was performed to remove missing or invalid values introduced during technical indicator computation. In addition to the raw market variables, a set of engineered features was created, including:

- Daily return
- Simple moving averages (SMA_10, SMA_50)
- Volatility measures
- MACD and MACD signal line values
- RSI values

The MACD was computed using exponential moving averages, and trading signals were generated when the MACD line crossed above or below its signal line. RSI values were used to identify momentum extremes, with buy signals assigned when the RSI dropped below 30 and sell signals when it rose above 70. The final label used for classification, `Final_Signal`, was created by combining the MACD and RSI actions such that a trade was only considered strong when both signals agreed. This design reduces noise while creating a more interpretable target variable.

For model training, categorical variables such as sector and industry were encoded using factorization to keep memory usage manageable. The target variable was encoded into numeric classes using

LabelEncoder, allowing the classifiers to work with a multi-class system: Hold, Buy, and Sell. The final feature set included price variables, engineered indicators, and encoded categorical fields.

Methodologies

Three machine learning approaches were implemented: Logistic Regression, Random Forests, and Support Vector Machines (SVMs).

Logistic Regression

Logistic Regression is a linear probabilistic model that estimates the likelihood of each class based on weighted input features. It is computationally efficient, straightforward to interpret, and often acts as a solid baseline in classification tasks. The model was trained on the training set and evaluated on the test set using accuracy and a full classification report.

This approach is especially useful for understanding whether the selected variables have a straightforward linear relationship with signal direction. However, it can struggle when the relationship between stock features and trading labels is highly nonlinear or when the class distribution is imbalanced.

Random Forests

Random Forest is an ensemble learning method that constructs many decision trees and aggregates their predictions. This approach captures nonlinear interactions between variables such as price momentum, volatility, and indicator thresholds. It is less sensitive to scaling and can model complex patterns without requiring heavy preprocessing.

In this project, Random Forests were trained with multiple estimators and evaluated with accuracy and classification reports. Their ensemble nature makes them robust for noisy financial data, but they can also overfit if trees grow too deeply or if the dataset contains highly correlated features.

Support Vector Machines (SVMs)

Support Vector Machines aim to find an optimal hyperplane that separates different classes while maximizing the margin between them. SVMs are effective in high-dimensional spaces and are often strong on structured tabular data, including classification tasks based on engineered financial features. However, they can be computationally expensive on large datasets, especially when using a non-linear kernel or large training sets.

To manage runtime constraints, the SVM was trained on a reduced subset of the data rather than the full training population. This was a practical compromise to keep the experiment feasible while still evaluating the model's suitability for this task.

Results and Evaluation

Each model was assessed using accuracy and a classification report, which included precision, recall, and F1-score for each signal class. These metrics are particularly important in a trading context because misclassifying a buy signal as a hold or a sell signal can have materially different consequences.

The models were evaluated on a chronological test set to preserve the temporal structure of the data and avoid information leakage from future observations. This is particularly important in financial forecasting, where a random split could create unrealistic results.

Model Comparison

- Logistic Regression: served as a reliable baseline and provided interpretable coefficients, but it may be limited by the linear assumptions of the model.
- Random Forest: generally better suited for capturing nonlinear interactions in technical indicators and price features, making it a stronger candidate for trading-signal classification.
- SVM: can achieve strong separation when properly tuned and when the data is well-suited to margin-based classification, but training cost and scalability were notable concerns in this project.

The classification report helps identify whether the models are stronger at predicting one type of signal than another. In practice, the buy and sell classes may be harder to detect than the hold class because market movements are often more subtle and imbalanced. Strong model performance depends not just on overall accuracy, but also on precision and recall for minority classes.

Model Optimization

Hyperparameter tuning was also explored to improve model performance. For the Random Forest model, a GridSearchCV procedure was applied with a parameter grid containing:

- `n_estimators`: [50, 100]
- `max_depth`: [5, 10, None]

This tuning process used cross-validation to select the best-performing configuration while reducing the risk of overfitting to the training data. The best parameters were then applied to the model and evaluated again on the test set.

For Logistic Regression, cross-validation was used to estimate stability and generalization ability. The model was evaluated across multiple folds to understand whether its performance remained consistent under different data partitions. This is important in time-series classification, where even small distribution changes can affect model reliability.

The purpose of optimization was to improve predictive performance, reduce variance, and ensure that the model generalizes beyond the training set. In this domain, improvements in cross-validation scores are often more meaningful than marginal gains in training accuracy.

Insights into Trading Signals

The use of MACD and RSI as feature-based trading indicators provided a structured way to translate momentum and trend information into labels for supervised learning. MACD reflects the relationship between short- and long-term moving averages, while RSI indicates relative strength and overbought or oversold conditions.

The models revealed that technical indicators are useful but imperfect. Certain signals appear more reliable than others:

- Buy signals from strong bullish MACD crossovers often align with upward momentum and may be easier to model when the market trend is persistent.
- Sell signals are often more difficult to predict because bearish movements can be abrupt and may involve higher volatility or short-lived reversals.
- RSI-based extremes can capture momentum shifts, but they may generate false positives when the market remains range-bound or experiences sudden volatility spikes.

The combination of MACD and RSI improved the interpretability of the target variable by requiring both signals to agree before issuing a final buy or sell signal. This reduces noise, but it may also miss opportunities where only one indicator is strongly predictive. In other words, the models reveal that technical indicators are informative, but they are not universally reliable across all market regimes.

Limitations

Several limitations exist in this approach:

- Financial markets are affected by events and regimes that technical indicators alone may not capture.

- The models are trained on historical data and may not generalize well to changing market conditions.
- Signal labels based on MACD/RSI thresholds may overfit to specific parameter choices and may not be optimal across different time horizons.
- The dataset may be imbalanced, which can make minority classes underrepresented and harder to predict accurately.

Conclusion and Future Work

This project demonstrates that machine learning can be applied to generate trading signals using technical indicators and historical stock data. The combination of preprocessing, feature engineering, and model evaluation created a practical workflow for classifying buy, hold, and sell conditions. The resulting models showed that it is feasible to use technical indicators such as MACD and RSI to build predictive frameworks, although performance depends strongly on signal definition, feature quality, and model choice.

The most important lesson is that technical indicators are useful as input features, but they should not be viewed as standalone market predictors. A robust trading strategy likely requires a broader feature set, better class balancing, and more sophisticated validation methods.

Future work should explore:

- Deep learning models such as LSTMs or temporal convolutional networks
- Additional features such as sentiment scores, macroeconomic variables, or more advanced volatility measures
- Alternative look-back and look-ahead windows for MACD and RSI calculations
- More robust handling of class imbalance through resampling or weighting
- Walk-forward validation to better simulate real-world trading environments

Overall, the project provides a strong foundation for forecasting stock-market trading signals and highlights the trade-off between model interpretability, computational cost, and predictive performance. Further research can improve the decision quality and transform these signal-classification models into more reliable trading decision systems.